Full-Stack Web Development Complete Revision Guide

Internet & JavaScript • React & TypeScript • Node & Express • MongoDB & SQL • Git & MERN • Docker, Redis, Nginx & System Design

6-Week Curriculum | 81 Topics Covered | Equal-Depth Quick Reference

Table of Contents

Week 1 — Internet, Networking & JavaScript

Internet & Networking • JavaScript

Week 2 — React & TypeScript

React Fundamentals • React Hooks • Routing • TypeScript

Week 3 — Backend: Node.js, Express & Authentication

Node.js • Express.js • APIs & Postman • Authentication

Week 4 — Databases: MongoDB & SQL

MongoDB • CRUD Operations • SQL (Very Important)

Week 5 — Development Tools & MERN Architecture

Git & GitHub • Chrome DevTools & VS Code • MERN Architecture & Deployment

Week 6 — Interviewer-Favorite Concepts

Docker • Redis • Nginx • Load Balancing • Scaling & System Design Basics • CI/CD

Optional (After Everything) — WebSockets, AWS S3 & System Design

WebSockets • AWS S3 & Object Storage • System Design

Week 1 — Internet, Networking & JavaScript

Internet & Networking

How the Internet Works

- The internet is a global network of interconnected computers that communicate using standardized protocols (TCP/IP).
- Data is broken into small units called packets, each with source/destination info, sent independently, and reassembled at the destination.
- Routers forward packets between networks; ISPs (Internet Service Providers) connect end users to the wider internet backbone.
- TCP/IP model layers: Application -> Transport -> Internet -> Link. Each layer handles a specific responsibility (e.g., HTTP at Application, TCP at Transport).
- TCP (Transmission Control Protocol) guarantees reliable, ordered delivery; UDP is faster but unreliable (used in streaming, gaming).

Client-Server Architecture

- A client (browser, mobile app) sends requests; a server processes them and returns responses — a request/response model.
- Servers host resources (web pages, APIs, files) and listen on specific ports (e.g., 80 for HTTP, 443 for HTTPS).
- Multiple clients can connect to one server concurrently; servers must handle concurrency (threads, event loops, load balancing).
- Common architecture: Client (frontend) -> Server (backend/API) -> Database, often called the 3-tier architecture.
- Alternative models exist (peer-to-peer, serverless) but client-server is the dominant web model.

HTTP Request-Response Cycle

- Client sends an HTTP request containing: method, URL/path, headers, and optionally a body.
- Server parses the request, performs logic (DB queries, computation), and sends back a response with status code, headers, and body.
- Each HTTP request is stateless by default — the server doesn't remember previous requests unless state is explicitly maintained (cookies, sessions, tokens).
- Cycle: DNS lookup -> TCP/TLS handshake -> HTTP request sent -> server processes -> HTTP response returned -> connection closed or kept alive.
- Keep-Alive connections and HTTP/2 multiplexing reduce the overhead of repeated handshakes for multiple requests.

HTTP Methods (GET, POST, PUT, PATCH, DELETE)

- GET: retrieve a resource; safe and idempotent; no request body (typically); data sent via query params.

- POST: create a new resource or submit data; not idempotent (calling twice may create two records); data sent in body.
- PUT: replace an entire resource with the provided data; idempotent (same result if repeated).
- PATCH: partially update a resource (only specified fields); not strictly guaranteed idempotent but usually treated as such.
- DELETE: remove a resource; idempotent (deleting an already-deleted resource has the same end state).

HTTP Status Codes

- 1xx Informational: request received, continuing process (rarely used directly, e.g., 100 Continue).
- 2xx Success: 200 OK, 201 Created, 204 No Content — request succeeded.
- 3xx Redirection: 301 Moved Permanently, 302 Found (temporary redirect), 304 Not Modified (caching).
- 4xx Client Errors: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity.
- 5xx Server Errors: 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable — fault lies with the server.

DNS for Developers

- DNS (Domain Name System) translates human-readable domain names into machine-readable IP addresses.
- It works like a distributed phonebook spread across many servers worldwide rather than one central database.
- Key record types: A (maps to IPv4), AAAA (IPv6), CNAME (alias to another domain), MX (mail server), TXT (verification/metadata), NS (nameservers).
- DNS uses caching at multiple levels (browser, OS, ISP resolver) to speed up repeated lookups; TTL (Time To Live) controls cache duration.
- Developers interact with DNS when configuring custom domains, SSL certificates, subdomains, and CDNs.

How a Domain Resolves to an IP Address

- Browser first checks its local cache, then OS cache, then checks the hosts file for the domain.
- If not cached, it queries a DNS resolver (often provided by the ISP or a public one like 8.8.8.8 / 1.1.1.1).
- Resolver queries the Root server -> Top-Level Domain (TLD) server (e.g., .com) -> Authoritative nameserver for the exact domain.
- The authoritative server returns the IP address, which is cached and returned to the browser.
- Browser then opens a TCP connection to that IP address to begin the HTTP(S) request.

IPv4 vs IPv6

- IPv4 uses 32-bit addresses (e.g., 192.168.1.1), giving about 4.3 billion unique addresses — now largely exhausted.
- IPv6 uses 128-bit addresses (e.g., 2001:0db8::1), providing a vastly larger address space to support more devices.
- IPv6 simplifies header structure, improves routing efficiency, and natively supports features like auto-configuration.
- NAT (Network Address Translation) was widely used to extend IPv4's limited address space by sharing one public IP among many devices.
- Both coexist today; many networks run dual-stack (supporting IPv4 and IPv6 simultaneously) during the transition.

HTTPS, SSL & TLS

- HTTPS is HTTP layered over TLS (Transport Layer Security), encrypting data between client and server.
- SSL is the predecessor to TLS; the term 'SSL' is still used colloquially, but modern systems use TLS (1.2/1.3).
- The TLS handshake establishes a shared secret key using asymmetric cryptography, then switches to faster symmetric encryption for the session.
- Certificates (issued by a Certificate Authority) verify a server's identity and contain its public key, preventing impersonation.
- HTTPS protects against eavesdropping, tampering, and man-in-the-middle attacks; browsers flag non-HTTPS sites as 'Not Secure'.

Cookies vs Session vs Local Storage

- Cookies: small key-value pairs sent with every HTTP request automatically; can have expiry, domain scope, and HttpOnly/Secure flags.
- Sessions: server-side storage of user state, typically referenced by a session ID stored in a cookie on the client.
- Local Storage: browser-based key-value storage (per origin), persists even after the browser closes, NOT sent automatically with requests.
- Session Storage: similar to Local Storage but cleared when the tab/browser closes; scoped to a single tab.
- Choosing between them depends on need: auth tokens often use cookies (HttpOnly) for security; UI preferences often use Local Storage.

JSON Fundamentals

- JSON (JavaScript Object Notation) is a lightweight, text-based data format for exchanging structured data.
- Syntax: objects use {key: value} pairs, arrays use [item1, item2], keys must be strings in double quotes.
- Valid value types: string, number, boolean, null, object, array (no functions, dates, or undefined).
- JSON is language-independent but maps naturally to JavaScript objects, making it the standard format for REST APIs.
- JS provides JSON.stringify() to convert objects to JSON strings, and JSON.parse() to convert JSON strings back into objects.

JavaScript

JavaScript Fundamentals (ES6+)

- let and const replaced var for block-scoped variables; const prevents reassignment (but not mutation of objects/arrays).
- Arrow functions (() => {}) provide concise syntax and do not bind their own 'this' (they inherit it from the enclosing scope).
- Template literals (`Hello ${name}`) allow embedded expressions and multi-line strings.
- Destructuring lets you unpack values from arrays/objects directly into variables: const {name, age} = user.
- Spread/rest operators (...) expand iterables into elements or gather multiple elements into an array/object.

- Modules (import/export) allow splitting code across files with explicit dependency declarations.

Objects, Arrays & Functions

- Objects store key-value pairs; accessed via dot notation (obj.key) or bracket notation (obj['key']).
- Arrays are ordered lists; common methods include map(), filter(), reduce(), forEach(), find(), and sort().
- Functions are first-class citizens — they can be assigned to variables, passed as arguments, and returned from other functions.
- Function declarations are hoisted; function expressions and arrow functions are not (they follow normal variable hoisting rules).
- Closures occur when a function 'remembers' variables from its enclosing scope even after that scope has finished executing.

JavaScript Promises

- A Promise represents a value that may be available now, later, or never — it has three states: pending, fulfilled, rejected.
- Created with new Promise((resolve, reject) => {...}); resolve() fulfills it, reject() rejects it.
- Consumed using .then() for success, .catch() for errors, and .finally() for cleanup that runs regardless of outcome.
- Promise.all() waits for all promises to resolve (fails fast if any reject); Promise.race() resolves/rejects as soon as one settles.
- Promises solve 'callback hell' by allowing chained, readable asynchronous code instead of deeply nested callbacks.

Async/Await

- async/await is syntactic sugar over Promises, making asynchronous code look synchronous and easier to read.
- An async function always returns a Promise; await pauses execution inside that function until the Promise settles.
- Error handling uses try/catch blocks around await calls, replacing .catch() chains.
- await can only be used inside an async function (or at the top level of ES modules).
- Multiple independent async calls should be run with Promise.all() rather than sequential awaits, to avoid unnecessary delays.

The JavaScript Event Loop

- JavaScript is single-threaded but achieves concurrency via the event loop, which manages execution order of code, events, and callbacks.
- The Call Stack executes synchronous code; Web APIs (timers, DOM events, fetch) handle async operations in the background.
- Completed async tasks are placed in queues: the Microtask Queue (Promises, async/await) and the Macrotask/Callback Queue (setTimeout, events).
- Microtasks are always processed before the next macrotask — meaning Promise callbacks run before setTimeout callbacks even if scheduled earlier.
- The event loop continuously checks: is the call stack empty? If so, pull the next task from the microtask queue (fully drain it), then one macrotask.

Callbacks vs Promises vs Async/Await

- Callbacks: functions passed as arguments to be executed later; simple but lead to deeply nested 'callback hell' for sequential async operations.
- Promises: an object-based abstraction over callbacks, enabling chaining via .then()/.catch() for flatter, more readable async code.
- Async/Await: built on Promises, allows writing async logic with synchronous-looking syntax, the most readable of the three approaches.
- All three ultimately rely on the same underlying event loop and Web APIs — async/await doesn't change how JS executes, just how it reads.
- Modern codebases favor async/await for clarity but still need to understand Promises since await operates on them under the hood.

Week 2 — React & TypeScript

React Fundamentals

React Basics

- React is a JavaScript library for building user interfaces using a component-based architecture.
- It uses a Virtual DOM — an in-memory representation of the UI — to efficiently calculate and apply only the necessary changes to the real DOM.
- JSX (JavaScript XML) lets you write HTML-like syntax directly in JavaScript, which gets compiled into React.createElement() calls.
- React apps are built from a tree of reusable components, each managing its own logic and rendering output.
- React re-renders components when state or props change, then efficiently 'diffs' the virtual DOM to update only changed elements.

Components, Props & State

- Components are functions (or classes) that return JSX; they can be reused with different inputs.
- Props (properties) are read-only inputs passed from a parent component to a child — they flow one-way (top-down).
- State is data managed within a component that can change over time and triggers a re-render when updated.
- Unlike props, state is local to the component and cannot be directly modified by a parent (unless passed down via callback functions).
- Lifting state up means moving shared state to the closest common parent so multiple children can access/modify it via props.

Functional Components

- Functional components are plain JavaScript functions that accept props and return JSX — the modern standard over class components.
- They are simpler, more concise, and (with Hooks) can manage state and side effects without needing 'this' binding.
- A functional component re-runs its entire function body on every re-render, recalculating JSX output based on current props/state.
- Conditional rendering is done with JS expressions (ternary operators, && operators, or early returns) inside the JSX.
- Keys are required when rendering lists of elements, helping React identify which items changed, were added, or removed.

React Hooks

useState

- useState(initialValue) returns a pair: the current state value and a setter function to update it.
- Calling the setter triggers a re-render of the component with the new state value.
- State updates can be asynchronous/batched — never rely on the state variable updating immediately after calling the setter.
- For updates depending on the previous value, pass a function to the setter: setCount(prev => prev + 1), to avoid stale state bugs.
- Each call to useState creates an independent piece of state; components can have multiple useState calls for different values.

useEffect

- useEffect(callback, dependencies) runs side effects (data fetching, subscriptions, DOM manipulation) after render.
- An empty dependency array [] means the effect runs only once, after the initial render (like componentDidMount).
- Omitting the dependency array means the effect runs after every render — usually not desired and can cause performance issues.
- Returning a cleanup function from useEffect runs before the component unmounts or before the effect re-runs (e.g., to clear timers/subscriptions).
- Including a dependency array with specific values re-runs the effect only when those values change between renders.

Other Common Hooks & Hook Rules

- useContext lets components read values from React Context without prop-drilling through every intermediate component.
- useRef creates a mutable reference that persists across renders without causing re-renders when changed (commonly used for DOM access).
- useMemo and useCallback memoize expensive computations and function references respectively, to optimize performance.
- Rules of Hooks: only call hooks at the top level of a component (not inside loops/conditions), and only call them from React functions.
- Custom hooks (functions prefixed with 'use') let you extract and reuse stateful logic across multiple components.

Routing

React Router DOM

- React Router enables client-side routing — navigating between views without a full page reload, preserving the SPA experience.
- Core components: <BrowserRouter> wraps the app, <Routes> contains <Route path='/...' element={...} /> definitions.
- <Link> and <NavLink> replace anchor tags for navigation without triggering a full browser reload.
- Dynamic segments (e.g., /users/:id) let routes match patterns; useParams() reads the matched values inside a component.

- useNavigate() programmatically redirects users (e.g., after form submission or login), and Outlet renders nested child routes.

Protected Routes

- Protected routes restrict access to certain pages unless the user meets a condition (usually being authenticated).
- Commonly implemented as a wrapper component that checks auth state and either renders the child route or redirects (e.g., to /login).
- The check often relies on context (AuthContext) or a stored token (e.g., in localStorage or cookies) to determine login status.
- <Navigate to='/login' /> is used inside the protected wrapper to redirect unauthenticated users declaratively.
- For role-based access, the same pattern extends to check user roles/permissions, not just authentication status.

TypeScript

TypeScript Fundamentals

- TypeScript is a superset of JavaScript that adds static typing, catching type errors at compile-time rather than runtime.
- Basic types include string, number, boolean, array (string[]), tuple, enum, any, unknown, void, and never.
- Interfaces and type aliases define the shape of objects: interface User { name: string; age: number }.
- Function parameters and return values can be typed explicitly, improving autocomplete and reducing bugs from incorrect usage.
- TypeScript compiles (transpiles) down to plain JavaScript using the TypeScript compiler (tsc) before running in browsers/Node.

TypeScript for React Developers

- Component props are typed using an interface or type, e.g., function Card(props: CardProps), ensuring correct usage at compile time.
- useState can be given an explicit generic type: useState<number>(0), useful when the initial value doesn't fully convey the type (e.g., null states).
- Event handler types come from React's built-in types, e.g., React.ChangeEvent<HTMLInputElement> for input onChange handlers.
- Children props are typically typed as React.ReactNode to allow any valid renderable content to be passed in.
- Generics allow writing reusable, type-safe components/functions (e.g., a List<T> component that works for any item type).

Week 3 — Backend: Node.js, Express & Authentication

Node.js

Node.js Fundamentals

- Node.js is a JavaScript runtime built on Chrome's V8 engine, allowing JS to run outside the browser (on servers).
- It provides built-in modules (fs, http, path, os, crypto, etc.) for tasks like file I/O, networking, and process management.
- npm (Node Package Manager) manages third-party packages; package.json tracks project dependencies and scripts.
- Node uses CommonJS (require/module.exports) by default, though ES Modules (import/export) are also supported with config.
- Node is non-blocking and asynchronous by design, making it well-suited for I/O-heavy applications like APIs and real-time apps.

Node.js Event Loop

- Node uses a single-threaded event loop (via libuv) to handle many concurrent operations without creating a thread per request.
- Phases of the event loop include: Timers, Pending Callbacks, Poll (I/O), Check (setImmediate), and Close Callbacks.
- Blocking/CPU-heavy operations on the main thread stall the entire event loop — heavy computation should be offloaded (worker threads, child processes).
- process.nextTick() and Promises (microtasks) run before the next event loop phase, giving them higher priority than setTimeout/setImmediate.
- Behind the scenes, libuv uses a thread pool for certain operations (file system, DNS) so they don't block the main thread.

Node.js Architecture

- Node's architecture combines the V8 engine (executes JS), libuv (handles async I/O and the event loop), and core/bindings (C++ APIs exposed to JS).
- Single-threaded event loop handles request dispatching, while expensive I/O tasks are delegated to libuv's thread pool behind the scenes.
- Node is designed around non-blocking I/O — instead of waiting for a file read or DB call, it registers a callback and continues processing other work.
- This makes Node highly efficient for I/O-bound apps (APIs, real-time chat) but less ideal for CPU-bound tasks (heavy computation, image processing).
- Clustering and worker threads allow Node apps to use multiple CPU cores, since a single Node process only uses one by default.

Express.js

Express.js Fundamentals

- Express is a minimal, flexible Node.js web framework for building servers and APIs with less boilerplate than raw Node's http module.
- A basic server: const app = express(); app.listen(PORT) — then define routes with app.get(), app.post(), etc.
- Express supports middleware functions that process requests before reaching route handlers (e.g., parsing JSON, logging, authentication).
- Built-in middleware like express.json() parses incoming JSON request bodies into req.body automatically.
- Express apps are typically organized into routes, controllers, and middleware folders for maintainability at scale.

Express Middleware

- Middleware functions have access to (req, res, next) and can modify the request/response or end the cycle before reaching the route handler.
- Calling next() passes control to the next middleware/route handler in the chain; forgetting it causes the request to hang.
- Middleware can be applied globally (app.use()), to specific routes, or chained in sequence for a single route.
- Error-handling middleware takes four arguments (err, req, res, next) and is typically defined last to catch errors from earlier middleware/routes.
- Common custom middleware use-cases: authentication checks, request logging, input validation, and rate limiting.

Routes & Controllers

- Routes define the URL endpoints and HTTP methods an app responds to (e.g., app.get('/users', getUsers)).
- Controllers contain the actual logic for handling a request — separating routing definitions from business logic improves maintainability.
- Express Router (express.Router()) allows grouping related routes into separate modules, then mounting them with app.use('/api/users', userRouter).
- Route parameters (req.params), query strings (req.query), and request body (req.body) are the main ways data flows into a route handler.
- A clean structure typically follows: Routes -> Controllers -> Services/Models, separating concerns at each layer.

APIs & Postman

Building REST APIs

- REST (Representational State Transfer) is an architectural style where resources are represented by URLs and manipulated via HTTP methods.
- A typical REST API maps CRUD operations to HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).

- Resources are usually pluralized nouns in URLs (e.g., /api/products, /api/products/:id) rather than verbs.
- Responses typically use JSON, with consistent structure (status, data, message/error) to simplify client-side handling.
- Proper status codes should be returned for every response (e.g., 201 for creation, 404 for not found) rather than always returning 200.

REST API Best Practices

- Use nouns for resource endpoints, not actions (e.g., GET /orders rather than GET /getOrders).
- Version your API (e.g., /api/v1/...) to allow backward-incompatible changes without breaking existing clients.
- Implement proper error handling with descriptive, consistent error response formats and appropriate status codes.
- Support pagination, filtering, and sorting for endpoints returning large collections (e.g., ?page=2&limit=20).
- Validate and sanitize all incoming data; never trust client input directly in database queries or logic.

Postman for API Testing

- Postman is a GUI tool for sending HTTP requests and inspecting responses, used to test and document APIs without writing client code.
- Collections group related requests together; environments store variables (like base URLs, tokens) that can switch between dev/staging/prod.
- Requests can include headers, query params, body data (JSON, form-data), and authorization details (Bearer tokens, Basic Auth).
- Postman supports automated tests via JavaScript snippets in the 'Tests' tab, checking response status, body content, and timing.
- Postman can generate code snippets for requests in multiple languages, and export collections for sharing or CI/CD integration.

Authentication

Authentication vs Authorization

- Authentication verifies WHO a user is (e.g., logging in with username/password) — confirming identity.
- Authorization determines WHAT an authenticated user is allowed to do (e.g., admin vs regular user permissions).
- Authentication always happens first; authorization decisions are made based on the authenticated identity.
- Common authentication methods: session-based (cookies), token-based (JWT), OAuth (third-party login like Google/GitHub).
- Authorization is often implemented via roles (admin, user) or permissions (read, write, delete) checked via middleware.

JWT Authentication

- JWT (JSON Web Token) is a compact, self-contained token format used to securely transmit identity information between parties.

- A JWT has three parts separated by dots: Header (algorithm info), Payload (claims/user data), and Signature (verifies integrity).
- The signature is created using a secret key (HMAC) or public/private key pair (RSA), ensuring the token hasn't been tampered with.
- JWTs are stateless — the server doesn't need to store session data; it just verifies the token's signature on each request.
- Tokens typically have an expiration (exp claim); refresh tokens are used to issue new access tokens without re-login.

JWT Login Flow

- User submits credentials (email/password) to a login endpoint; server verifies them against the database.
- If valid, the server generates a signed JWT (often an access token + refresh token) and returns it to the client.
- The client stores the token (commonly in an HttpOnly cookie or memory/local storage) and includes it in the Authorization header on future requests.
- Protected routes use middleware to extract and verify the token's signature/expiry before allowing access to the resource.
- When the access token expires, the client uses the refresh token to request a new one without forcing the user to log in again.

Password Hashing with bcrypt

- Passwords must never be stored in plain text; bcrypt is a widely used hashing algorithm designed specifically for password security.
- bcrypt incorporates a 'salt' automatically, ensuring identical passwords produce different hashes, preventing rainbow-table attacks.
- It includes a configurable 'cost factor' (work factor) that controls how computationally expensive hashing is, slowing down brute-force attacks.
- Typical usage: bcrypt.hash(password, saltRounds) to hash on registration, and bcrypt.compare(input, storedHash) to verify on login.
- Because hashing is one-way, passwords can never be 'decrypted' — only verified by re-hashing the input and comparing.

Week 4 — Databases: MongoDB & SQL

MongoDB

MongoDB Fundamentals

- MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like documents (BSON internally).
- Data is organized into Collections (like tables) containing Documents (like rows), but documents in the same collection can have different fields.
- Unlike relational DBs, MongoDB doesn't enforce a fixed schema by default, allowing more flexibility during early development.
- Each document has a unique _id field, automatically generated as an ObjectId unless explicitly provided.
- MongoDB excels at horizontal scaling (sharding) and is well-suited to hierarchical/nested data and rapidly evolving schemas.

Mongoose

- Mongoose is an ODM (Object Data Modeling) library for MongoDB in Node.js, providing schema definitions and validation on top of raw MongoDB.
- A Schema defines the structure, types, and validation rules for documents (e.g., name: { type: String, required: true }).
- A Model is a compiled version of a schema, providing an interface (find, save, update, delete) to interact with the actual collection.
- Mongoose supports middleware (pre/post hooks) that run before or after operations like save() or remove(), useful for hashing passwords, etc.
- Built-in validation, default values, and custom validators help maintain data integrity despite MongoDB's flexible nature.

MongoDB Schema & Model Design

- Schema design in MongoDB favors denormalization (embedding related data) over normalization, optimizing for read performance.
- Embedding is used for tightly coupled, frequently accessed-together data (e.g., an order with embedded line items).
- Referencing (storing an ObjectId reference, similar to a foreign key) is used for data that's large, shared, or changes independently.
- Mongoose's populate() method resolves referenced ObjectIds into full documents, mimicking a SQL JOIN-like behavior.
- Choosing between embedding vs referencing depends on data size, update frequency, and query patterns — a key MongoDB design decision.

CRUD Operations

CRUD Fundamentals

- CRUD stands for Create, Read, Update, Delete — the four basic operations performed on persistent data.
- In MongoDB: Create uses insertOne()/insertMany(), Read uses find()/findOne(), Update uses updateOne()/updateMany(), Delete uses deleteOne()/deleteMany().
- In Mongoose: Model.create(), Model.find(), Model.findByIdAndUpdate(), and Model.findByIdAndDelete() are common convenience methods.
- CRUD operations typically map directly to REST API endpoints and their corresponding HTTP methods (POST, GET, PUT/PATCH, DELETE).
- Proper CRUD implementations include validation, error handling, and appropriate status codes/responses for each operation.

Building a CRUD API (Node + MongoDB)

- Typical stack: Express handles routing, Mongoose models define data shape, and MongoDB stores the actual documents.
- Flow: Route receives request -> Controller calls Mongoose model methods -> Model interacts with MongoDB -> Controller sends a response.
- Connecting to MongoDB is done once at app startup using mongoose.connect(connectionString), often with a URI from MongoDB Atlas (cloud) or local instance.
- Error handling should distinguish between validation errors (400), not-found errors (404), and unexpected server errors (500).
- Environment variables (.env file) are used to store sensitive config like the database connection string, keeping secrets out of source code.

SQL (Very Important)

SQL Fundamentals

- SQL (Structured Query Language) is used to interact with relational databases (MySQL, PostgreSQL, SQL Server, etc.) using structured tables.
- Core commands: SELECT (retrieve), INSERT (add), UPDATE (modify), DELETE (remove), and DDL commands like CREATE TABLE, ALTER TABLE.
- Relational databases enforce a fixed schema — every row in a table follows the same column structure and data types.
- WHERE clauses filter rows based on conditions; ORDER BY sorts results; LIMIT restricts the number of rows returned.
- Primary keys uniquely identify each row in a table; foreign keys reference primary keys in other tables to establish relationships.

SQL Joins

- JOINs combine rows from two or more tables based on a related column, typically a foreign key relationship.
- INNER JOIN: returns only rows where there is a match in both tables.
- LEFT JOIN (LEFT OUTER JOIN): returns all rows from the left table, plus matched rows from the right (NULLs where no match).
- RIGHT JOIN: mirror of LEFT JOIN — returns all rows from the right table, plus matches from the left.
- FULL OUTER JOIN: returns all rows from both tables, with NULLs where no match exists on either side.

GROUP BY and HAVING

- GROUP BY aggregates rows sharing a common value in specified columns, typically used with aggregate functions (COUNT, SUM, AVG, MAX, MIN).
- Example: SELECT department, COUNT(*) FROM employees GROUP BY department — counts employees per department.
- WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation (e.g., HAVING COUNT(*) > 5).
- You cannot use aggregate functions directly in a WHERE clause — that's precisely why HAVING exists.
- Columns in the SELECT list (that aren't aggregated) generally must appear in the GROUP BY clause as well.

Database Indexing

- An index is a data structure (commonly a B-Tree) that speeds up data retrieval by avoiding full table scans.
- Indexes are typically created on columns frequently used in WHERE clauses, JOIN conditions, or ORDER BY.
- Trade-off: indexes speed up reads but slow down writes (INSERT/UPDATE/DELETE) since the index must also be updated.
- A primary key is automatically indexed; composite indexes cover multiple columns for queries filtering on combinations of fields.
- Over-indexing wastes storage and degrades write performance, so indexes should be added deliberately based on actual query patterns.

SQL Interview Essentials

- Be ready to explain the difference between DELETE, TRUNCATE, and DROP (row removal vs table reset vs full table removal).
- Know the difference between a primary key (unique identifier, only one per table) and a unique key (also enforces uniqueness, multiple allowed).
- Understand normalization (organizing data to reduce redundancy: 1NF, 2NF, 3NF) versus denormalization (trading redundancy for read performance).
- Be able to write basic queries combining SELECT, JOIN, WHERE, GROUP BY, HAVING, and ORDER BY in a single statement.
- Know the difference between clustered and non-clustered indexes, and how transactions (COMMIT/ROLLBACK) ensure data consistency.

Week 5 — Development Tools & MERN Architecture

Git & GitHub

Git & GitHub Fundamentals

- Git is a distributed version control system that tracks changes to files over time, allowing you to revert, compare, and collaborate safely.
- Core commands: git init, git add, git commit, git status, git log, git push, and git pull.
- A repository (repo) contains the entire history of a project; commits are snapshots with a message describing the change.
- GitHub is a cloud platform for hosting Git repositories, adding collaboration features (issues, pull requests, actions) on top of Git.
- The staging area (via git add) lets you choose exactly which changes to include in the next commit, rather than committing everything at once.

Branching & Merging

- Branches allow parallel lines of development (e.g., feature branches) without affecting the main/production codebase.
- git branch creates a new branch; git checkout -b (or git switch -c) creates and switches to it in one step.
- Merging combines changes from one branch into another; a 'fast-forward' merge happens when there's no divergent history.
- Merge conflicts occur when the same lines were changed differently in both branches and must be resolved manually.
- git stash temporarily saves uncommitted changes so you can switch branches cleanly and reapply them later.

Git Rebase vs Merge

- Merge creates a new 'merge commit' that combines two branch histories, preserving the full original history including all branch commits.
- Rebase moves/replays your branch's commits on top of another branch's tip, creating a linear, cleaner-looking history.
- Rebase rewrites commit history (new commit hashes), so it should be avoided on shared/public branches that others have already pulled.
- Merge is generally safer for shared branches; rebase is often preferred for cleaning up a local feature branch before opening a PR.
- The golden rule: never rebase a branch that others are actively working on, since it can cause significant history conflicts for them.

GitHub Pull Requests & Collaboration

- A Pull Request (PR) proposes merging changes from one branch into another, enabling code review before integration.
- PRs display a diff of changes, allow inline comments, and often integrate with CI checks (tests, linting) that must pass before merging.
- Typical workflow: fork/branch -> commit changes -> push -> open PR -> review/discuss -> approve -> merge -> delete branch.
- Forking creates a personal copy of someone else's repository, commonly used for contributing to open-source projects you don't have write access to.
- GitHub Issues track bugs, feature requests, and tasks, and can be linked directly to PRs that resolve them.

Chrome DevTools & VS Code

Chrome DevTools

- DevTools (F12 or Ctrl+Shift+I) is a built-in suite for inspecting, debugging, and profiling web pages directly in the browser.
- The Elements panel inspects/edits live HTML and CSS; the Console runs JavaScript and shows logs/errors.
- The Network tab shows all requests made by the page (status, timing, headers, payload), essential for debugging API calls.
- The Sources panel allows setting breakpoints in JavaScript code to pause execution and step through logic line by line.
- The Application tab inspects storage mechanisms (cookies, Local Storage, Session Storage, IndexedDB) used by the page.

Debugging JavaScript with DevTools

- Breakpoints can be set directly in the Sources panel by clicking a line number, pausing execution when that line is reached.
- console.log(), console.error(), console.table(), and console.warn() provide different ways to inspect values during execution.
- The 'Watch' panel lets you track specific variable values as you step through paused code.
- The Call Stack panel shows the chain of function calls that led to the current paused point, useful for tracing bugs.
- Conditional breakpoints (right-click a line number) pause execution only when a specified expression evaluates to true.

VS Code Tips & Shortcuts

- Command Palette (Ctrl/Cmd+Shift+P) gives quick access to virtually every VS Code command without using the mouse.
- Multi-cursor editing (Alt+Click or Ctrl+D to select next match) allows editing multiple lines/instances simultaneously.
- Integrated terminal (Ctrl+`) lets you run commands without leaving the editor.
- Extensions (like ESLint, Prettier, GitLens) extend functionality for linting, formatting, and enhanced Git integration.

- Built-in debugger supports breakpoints and step-through debugging for Node.js and browser-based JavaScript directly inside the editor.

MERN Architecture & Deployment

MERN Stack Architecture

- MERN stands for MongoDB, Express, React, Node.js — a full JavaScript stack from database to frontend.
- React (frontend) runs in the browser, handling UI rendering and user interaction.
- Express + Node (backend) handles business logic, authentication, and exposes a REST API.
- MongoDB (database) persists application data as documents, accessed via Mongoose from the Express backend.
- Using JavaScript across the entire stack reduces context-switching and allows sharing logic/types (especially with TypeScript) between layers.

How Frontend, Backend & Database Work Together

- The React frontend sends HTTP requests (via fetch/axios) to specific API endpoints exposed by the Express backend.
- Express routes receive the request, validate/process it, and interact with MongoDB through Mongoose models as needed.
- The database returns data to Express, which formats it (usually as JSON) and sends it back as the HTTP response.
- React receives the JSON response, updates component state, and re-renders the UI to reflect the new data.
- This request-response cycle repeats for every user interaction that requires fresh or updated data from the server.

Deploying a MERN App

- Frontend (React build) is typically deployed to static hosts like Vercel, Netlify, or GitHub Pages.
- Backend (Node/Express) is typically deployed to platforms like Render, Railway, or a VPS — services that keep a server process running.
- MongoDB is often hosted via MongoDB Atlas (managed cloud database) rather than self-hosted, for ease of scaling and backups.
- CORS (Cross-Origin Resource Sharing) must be configured on the backend to allow the deployed frontend's domain to make API requests.
- Free-tier deployments are common for learning/portfolio projects, but may have cold-starts, sleep timers, or resource limits.

Environment Variables

- Environment variables store configuration values (API keys, DB URIs, secrets) outside of the source code.
- In Node.js, the dotenv package loads variables from a .env file into process.env at runtime.
- Sensitive values (DB passwords, JWT secrets) should never be hardcoded or committed to version control — .env should be in .gitignore.
- Different environments (development, staging, production) often use different sets of environment variables for the same variable names.

- Deployment platforms (Vercel, Render, etc.) provide a dashboard/CLI to securely set environment variables for live apps.

Week 6 — Interviewer-Favorite Concepts

Docker

Docker Fundamentals

- Docker packages an application with all its dependencies into a 'container' — a lightweight, portable, isolated execution unit.
- A Dockerfile defines how to build an image (base OS/runtime, copied files, install steps, start command).
- An image is a read-only template; a container is a running instance of that image.
- Common commands: docker build (create image), docker run (start container), docker ps (list running containers), docker stop.
- Docker Compose (docker-compose.yml) defines and runs multi-container applications (e.g., app + database) with a single command.

Docker in Practice

- Containers share the host OS kernel, making them much more lightweight and faster to start than full virtual machines.
- Images are built in layers; Docker caches unchanged layers, speeding up rebuilds when only some files change.
- Volumes persist data outside a container's lifecycle (e.g., database files), so data isn't lost when a container is removed.
- Docker networks allow containers to communicate with each other by name, useful for multi-service applications (app + db + cache).
- Container registries (like Docker Hub) store and distribute images, enabling consistent deployment across environments.

Docker Containers vs Virtual Machines

- VMs virtualize an entire hardware stack including a full guest OS; containers virtualize only at the OS level, sharing the host kernel.
- Containers are significantly lighter (MBs vs GBs) and start in seconds, whereas VMs can take minutes to boot.
- Because VMs include a full OS, they offer stronger isolation; containers offer weaker isolation but far better resource efficiency.
- Multiple containers can run on a single host OS, while each VM requires its own OS instance, consuming more memory/CPU overhead.
- In modern cloud-native deployments, containers (often orchestrated by Kubernetes) are favored for microservices and horizontal scaling.

Redis

Redis Fundamentals

- Redis (Remote Dictionary Server) is an in-memory key-value data store, used primarily for caching, sessions, and fast lookups.
- Because data lives in RAM rather than disk, Redis offers extremely fast read/write performance compared to traditional databases.
- Supports rich data structures beyond simple strings: lists, sets, sorted sets, hashes, and even streams/pub-sub messaging.
- Data can optionally be persisted to disk (snapshotting or append-only logs) to survive restarts, though Redis is primarily memory-first.
- Common use cases: caching frequent DB query results, session storage, rate limiting, leaderboards, and real-time pub/sub messaging.

Caching with Redis

- Caching stores frequently accessed data in a fast-access layer (Redis) to avoid repeatedly querying a slower database.
- Common pattern: check Redis cache first (cache hit = fast return); if not found (cache miss), query DB, then store result in Redis for next time.
- TTL (Time To Live) is set on cached keys so stale data automatically expires after a defined period.
- Cache invalidation (updating/removing cached data when the underlying data changes) is one of the hardest problems in caching strategy.
- Redis caching significantly reduces database load and improves response times for read-heavy applications.

Nginx

Nginx Fundamentals

- Nginx is a high-performance web server, commonly used to serve static files, act as a reverse proxy, and handle load balancing.
- It uses an event-driven, asynchronous architecture, allowing it to efficiently handle thousands of concurrent connections.
- Configuration is defined in nginx.conf using 'server blocks' that specify how to handle requests for particular domains/ports.
- Nginx is widely used to terminate SSL/TLS (handle HTTPS) before forwarding plain HTTP requests internally to application servers.
- It also commonly handles gzip compression, caching static assets, and serving as the entry point for an entire infrastructure stack.

Nginx as a Reverse Proxy

- A reverse proxy sits in front of backend servers, forwarding client requests to the appropriate server and returning the response.

- This hides backend server details from clients, adding a layer of security and enabling flexible backend changes without affecting clients.
- Configured via 'proxy_pass' directives, e.g., forwarding requests at /api to an internal Node.js server running on a different port.
- Enables a single public-facing domain to route to multiple internal services (e.g., /api -> backend, / -> frontend build).
- Reverse proxies also enable load balancing, SSL termination, and centralized logging/security policies across multiple backend instances.

Load Balancing

Load Balancer Fundamentals

- A load balancer distributes incoming traffic across multiple servers, preventing any single server from becoming overwhelmed.
- It improves availability (if one server fails, traffic reroutes to healthy ones) and scalability (handle more traffic by adding servers).
- Common algorithms: Round Robin (cycles through servers evenly), Least Connections (sends to the server with fewest active connections), IP Hash (consistent routing based on client IP).
- Load balancers can operate at Layer 4 (transport, e.g., TCP/UDP) or Layer 7 (application, e.g., HTTP-aware routing based on URL paths/headers).
- Health checks periodically ping backend servers to detect failures and automatically remove unhealthy servers from the rotation.

Load Balancing in Practice

- Nginx, HAProxy, and cloud-native load balancers (AWS ELB, GCP Load Balancer) are common tools/services used to implement load balancing.
- Sticky sessions route a specific client's requests to the same backend server, useful when session data is stored locally on that server.
- Load balancers are often combined with auto-scaling — automatically adding/removing server instances based on real-time traffic load.
- A load balancer is itself a single point of failure unless made redundant (e.g., multiple load balancer instances behind a DNS failover).
- Global load balancing can route users to the geographically nearest data center, reducing latency for distributed user bases.

Scaling & System Design Basics

Horizontal vs Vertical Scaling

- Vertical scaling ('scaling up') means adding more resources (CPU, RAM) to a single existing server.
- Horizontal scaling ('scaling out') means adding more servers/instances and distributing load across them (often via a load balancer).

- Vertical scaling has hardware limits and a single point of failure; horizontal scaling offers better fault tolerance and theoretically limitless growth.
- Horizontal scaling typically requires stateless application design (no server-specific session data) so any instance can handle any request.
- Databases are harder to scale horizontally than stateless app servers, often requiring sharding, replication, or specialized distributed DB systems.

System Design Basics for Scaling

- Caching (e.g., Redis) reduces repeated load on databases by serving frequent requests from fast in-memory storage.
- Database replication creates copies of data across multiple servers, improving read throughput and providing failover redundancy.
- Sharding splits a large dataset across multiple database instances based on a key, allowing horizontal scaling of storage and write throughput.
- CDNs (Content Delivery Networks) cache and serve static assets from servers geographically closer to users, reducing latency.
- Designing for scale generally means embracing statelessness, caching aggressively, and decoupling components (e.g., via message queues).

CI/CD

CI/CD Fundamentals

- CI (Continuous Integration) is the practice of frequently merging code changes into a shared repository, with automated builds/tests on each change.
- CD (Continuous Delivery/Deployment) automates releasing tested code to staging or production environments after CI passes.
- A typical pipeline: code pushed -> automated tests run -> build artifact created -> deploy to staging -> (manual or automatic) deploy to production.
- CI/CD reduces manual errors, catches bugs earlier, and enables faster, more frequent, more reliable releases.
- Pipelines are usually defined declaratively in config files (e.g., YAML) stored alongside the code in the repository.

GitHub Actions for CI/CD

- GitHub Actions automates workflows directly within a GitHub repository, triggered by events like pushes, pull requests, or schedules.
- Workflows are defined in YAML files under .github/workflows/, specifying jobs, steps, and the runner environment.
- A typical workflow: checkout code -> install dependencies -> run linter/tests -> build -> deploy (e.g., to Vercel, AWS, or Docker registry).
- Secrets (API keys, deployment tokens) are stored securely in GitHub repository settings and referenced in workflows without exposing them in code.
- Matrix builds allow testing across multiple environments (Node versions, OS) in parallel within a single workflow definition.

Optional (After Everything) — WebSockets, AWS S3 & System Design

WebSockets

WebSockets Fundamentals

- WebSockets provide a persistent, full-duplex connection between client and server, unlike HTTP's request-response-then-close model.
- Once established (via an HTTP 'upgrade' handshake), both client and server can send messages to each other at any time, in real time.
- Ideal for use cases requiring live updates: chat apps, live notifications, collaborative editing, multiplayer games, and live dashboards.
- Unlike repeated HTTP polling, WebSockets avoid the overhead of constantly opening new connections, reducing latency and server load.
- The connection stays open until explicitly closed by either side, requiring careful handling of reconnects and connection state.

Socket.IO

- Socket.IO is a library built on top of WebSockets (with fallbacks) that simplifies real-time, bidirectional communication in JS apps.
- It provides automatic reconnection, fallback to HTTP long-polling if WebSockets aren't available, and built-in event-based messaging.
- Core pattern: socket.emit('eventName', data) sends an event; socket.on('eventName', callback) listens for and handles it.
- Supports 'rooms' and 'namespaces' to group connected clients, useful for features like separate chat channels or private messaging.
- Requires both a server-side (Node.js) and client-side (browser) library working together to establish and manage connections.

AWS S3 & Object Storage

AWS S3 Fundamentals

- Amazon S3 (Simple Storage Service) is a scalable cloud object storage service for storing and retrieving any amount of data.
- Data is organized into 'buckets' (top-level containers) holding 'objects' (files), each identified by a unique key (like a file path).
- S3 is commonly used for storing user uploads (images, videos, documents), static website hosting, and backups.

- Access control is managed via IAM policies and bucket policies, controlling who can read/write specific buckets or objects.
- S3 offers different storage classes (Standard, Infrequent Access, Glacier) trading off cost vs retrieval speed for different access patterns.

Object Storage Concepts

- Object storage stores data as discrete 'objects' (data + metadata + unique ID) rather than as files in a folder hierarchy or blocks on a disk.
- This makes it highly scalable and well-suited for unstructured data (images, videos, backups, logs) accessed over HTTP(S) APIs.
- Unlike traditional file storage, object storage has no real folder structure — apparent 'folders' are just naming conventions in the object key.
- Object storage is typically accessed via REST APIs (e.g., S3's API), making it easy to integrate into web/mobile applications directly.
- It's the standard choice for storing user-generated content in modern web apps, since it scales without manual disk management.

System Design

System Design Fundamentals

- System design is the process of defining the architecture, components, and data flow of a system to meet functional and scalability requirements.
- Key considerations: scalability, availability, reliability, latency, consistency, and cost trade-offs.
- The CAP theorem states a distributed system can only guarantee two of three: Consistency, Availability, and Partition tolerance.
- Common building blocks: load balancers, caches, databases (SQL/NoSQL), message queues, and CDNs, combined based on the system's needs.
- Good system design interviews focus on clarifying requirements first, then iteratively designing and justifying trade-offs.

System Design Interview Basics

- Start by clarifying requirements: functional (what it does) and non-functional (scale, latency, availability expectations).
- Estimate scale early (requests per second, storage needs) to inform architecture decisions (e.g., do you need sharding/caching?).
- Sketch a high-level architecture first (client -> load balancer -> servers -> cache -> database), then dive deeper into specific components.
- Discuss trade-offs explicitly (e.g., SQL vs NoSQL, consistency vs availability) rather than presenting one 'correct' answer.
- Be ready to discuss bottlenecks and how you'd address them (e.g., adding caching, read replicas, or message queues for async processing).

Case Study: Designing a URL Shortener

- Core requirement: given a long URL, generate a short unique alias that redirects to the original when visited.
- Short code generation options: base62 encoding of an auto-incrementing ID, or a hash of the URL (with collision handling).
- Data model: a simple table/collection mapping short_code -> original_url, plus metadata (creation date, click count, expiry).
- On redirect requests, the service looks up the short code (often from a cache like Redis first) and issues an HTTP redirect (301/302) to the original URL.
- Scaling considerations: heavy read traffic favors caching popular redirects; the ID-generation service must avoid collisions even under high concurrency.